香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)
信息枢纽
INFORMATION HUB
人工智能学域
ARTIFICIAL INTELLIGENCE

# End-to-End Navigation I

Course AIAA 4220

Week 10- Lecture 19

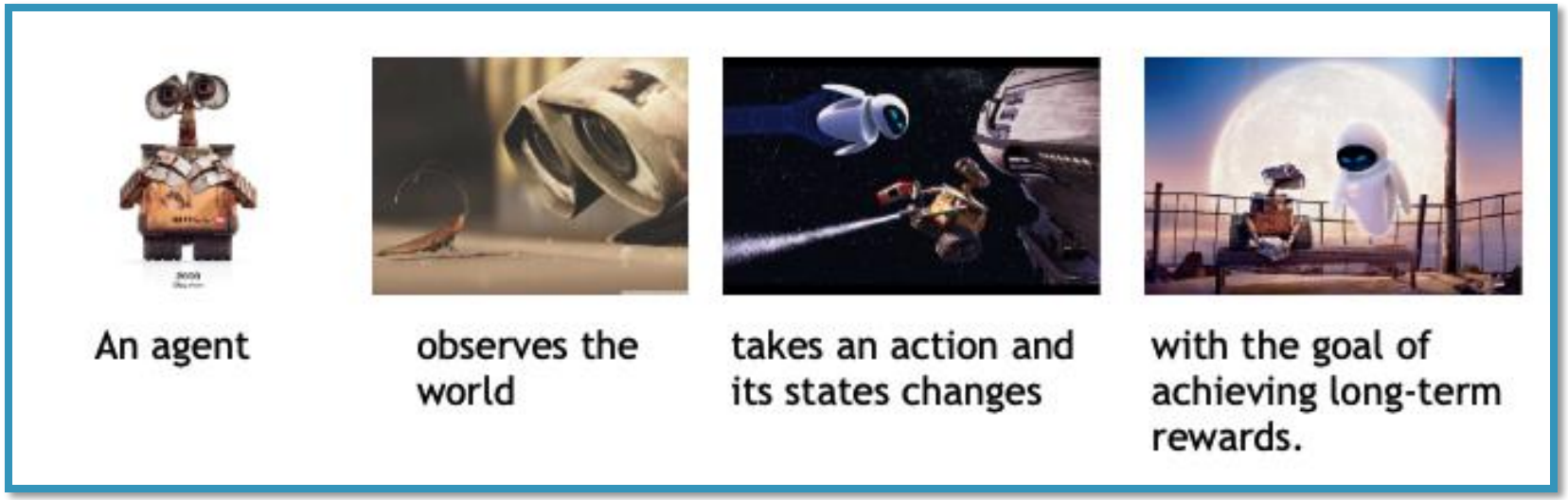
**Changhao Chen**

Assistant Professor

HKUST (GZ)

# RL Problem

- In RL, we typically focus on **sequential decision making**: an agent chooses a sequence of actions which each affect future possibilities available to the agent.

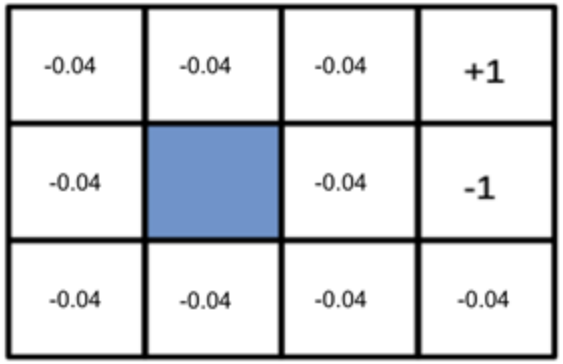


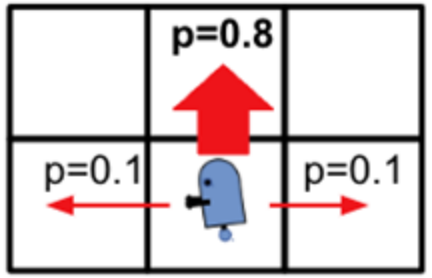

- What makes reinforcement learning different from other machine learning paradigms?
  - There is no supervisor, only a **reward signal**
  - **Feedback is delayed, not instantaneous**
    - **That means we need to run the policy network for a number of steps, then do gradient descent**
  - Time really matters (sequential, non i.i.d data)
  - Agent's actions affect the subsequent data it receives

# MDP Definition

- Given a sequential decision problem in a fully observable, stochastic environment with a known **Markovian transition model**
- Then a Markov Decision Process (MDP) is defined by the components
  - Set of states: *S*
  - Set of actions: *A*
  - Initial state: $s_0$
  - **Transition model**: *T(s, a, s')*
  - **Reward function**: *R(s) / R(s, a, s')*
  - An MDP is a 4-tuple: *M = (S, A, T, R)*

For reinforcement learning, the transition model and/or the reward function need to be learned



# Comparison

- Both value iteration and policy iteration compute the same thing (all optimal values)
- In value iteration:
  - Every iteration updates both the values and (implicitly) the policy
  - We don’t track the policy, but taking the max over actions implicitly recomputes it
- In policy iteration:
  - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
  - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
  - The new policy will be better (or we’re done)
- Both are dynamic programs for solving MDPs

# Policy

- A policy is the agent's behaviour
- It is a mapping from state to action, e.g.
  - Deterministic policy: $a = \pi(s)$
  - Stochastic policy: $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$

# Value Function

- Value function is a prediction of future reward
- Used to evaluate the goodness/badness of states
- And therefore to select between actions, e.g.

$$v_\pi(s) = \mathbb{E}_\pi \left[ R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots \mid S_t = s \right]$$

Expectation of the future cumulative rewards

# Model

- A model predicts what the environment will do next
  - Namely, the **state transition model** and **reward function**
- $\mathcal{P}$ predicts the next state
  - Previously in MDP it is represented as: $T(s, a, s')$

- $\mathcal{R}$ predicts the next (immediate) reward

$$\mathcal{P}^a_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$$
$$\mathcal{R}^a_s = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$$

# Towards Learning

- Now let's focus on reinforcement learning, where the environment is unknown. How can we apply learning?
  - 1. Learn a model of the environment (state transition model & reward function), and do planning in the model (i.e. **model-based reinforcement learning**)
    - You already know how to do this in principle (solving a MDP), but it's very hard to get to work.
  - 2. Learn a **value function** (e.g. **Q-learning**)
  - 3. Learn a **policy** directly (e.g. **policy gradient**)
- How can we deal with extremely large state spaces?
  - **Function approximation**: choose a parametric form for the policy and/or value function (e.g. linear in features, neural net, etc.)

# Summary of RL methods

- **Value Based**

Value iteration
Policy iteration
(Deep) Q-learning

- Learned Value Function
- Implicit policy (e.g. ε-greedy)

- **Policy Based**

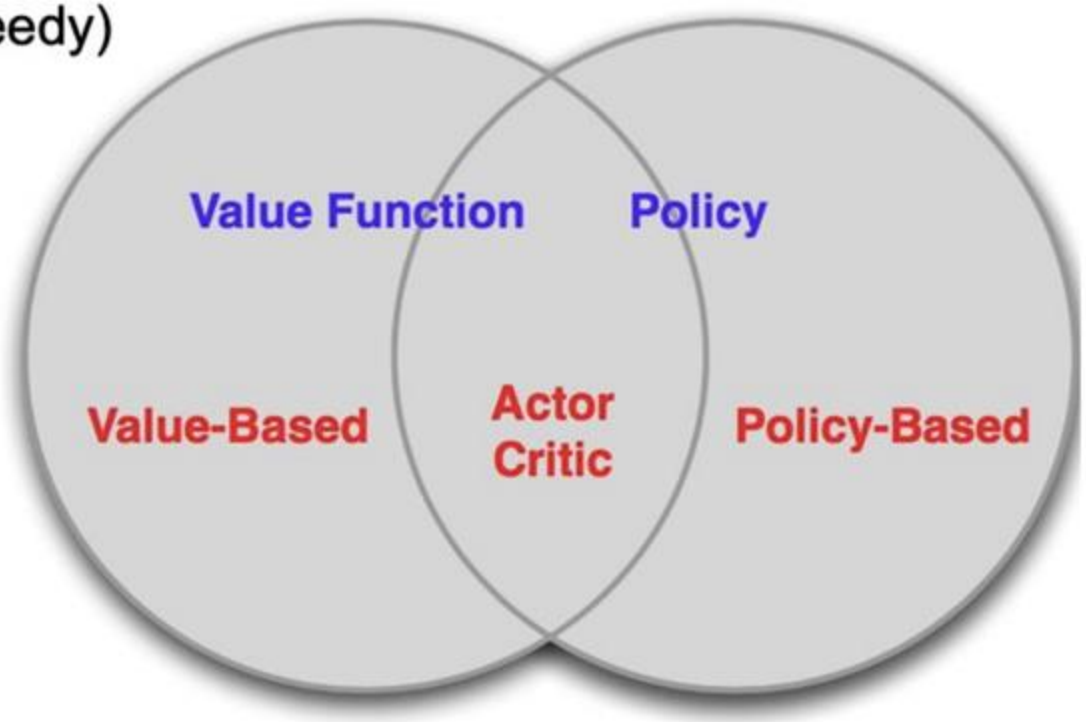
Policy gradients

- No Value Function
- Learned Policy

- **Actor-Critic**

Actor (policy)
Critic (Q-values)

- Learned Value Function
- Learned Policy




* credit: Fragkiadaki

# Traditional Robotics Navigation

**Pipeline:**
**Perception → Mapping → Planning → Control**

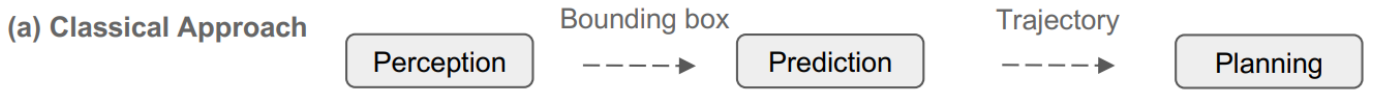
**Components**
•SLAM / VIO (pose estimation)
•Map building (grid / mesh / semantic)
•Motion planning (A*, RRT*)
•Low-level control (PID, MPC)

**Pros**
•Interpretable, stable, safe

**Cons**
•Complex tuning, brittle to failures
•Error accumulation across modules

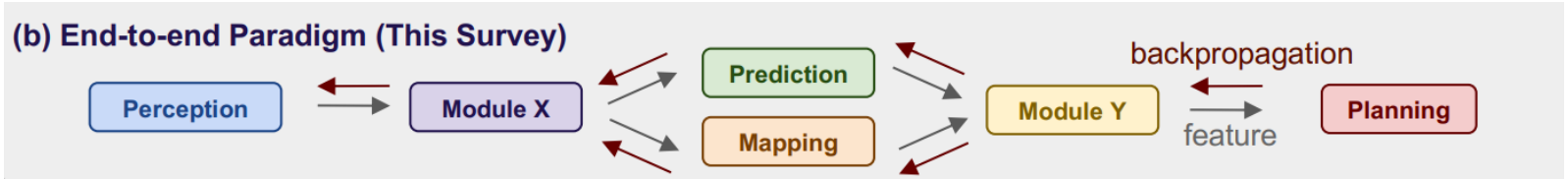
# Why End-to-End?

**Traditional Modular Navigation:**




vs.

**End-to-End Navigation:**




**Advantages:**

1. Simplicity in combining perception, prediction, and planning
2. Optimization of system and intermediate representations
3. Computational efficiency through shared backbones
4. Data-driven optimization via resource scaling

# End-to-End Navigation

**Definition**

***Direct mapping from raw sensory input to navigation actions***

**Inputs**

- Camera, LiDAR, IMU, GPS
- Language (VLN tasks)

**Outputs**

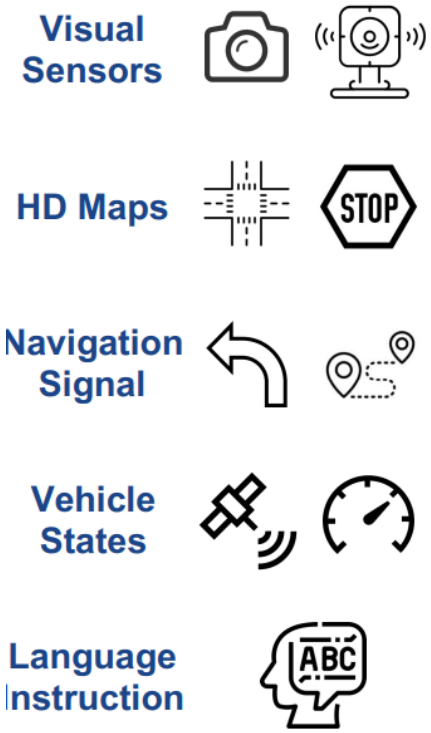
- Low-level control (steering, throttle)
- Waypoints / future trajectory
- Affordances (traffic lights, drivable area)

| Modality | Example |
|---|---|
| RGB cameras | end-to-end perception |
| LiDAR | accurate depth, 3D space |
| IMU | motion cues |
| GPS | global grounding |

# End-to-End Navigation

**Data Flow**

**Input:**

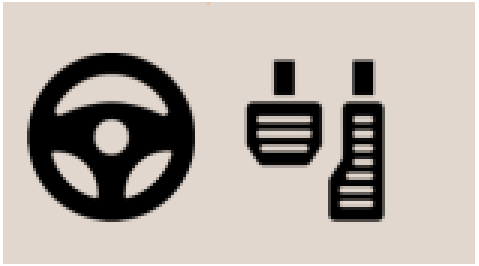



**Output:**

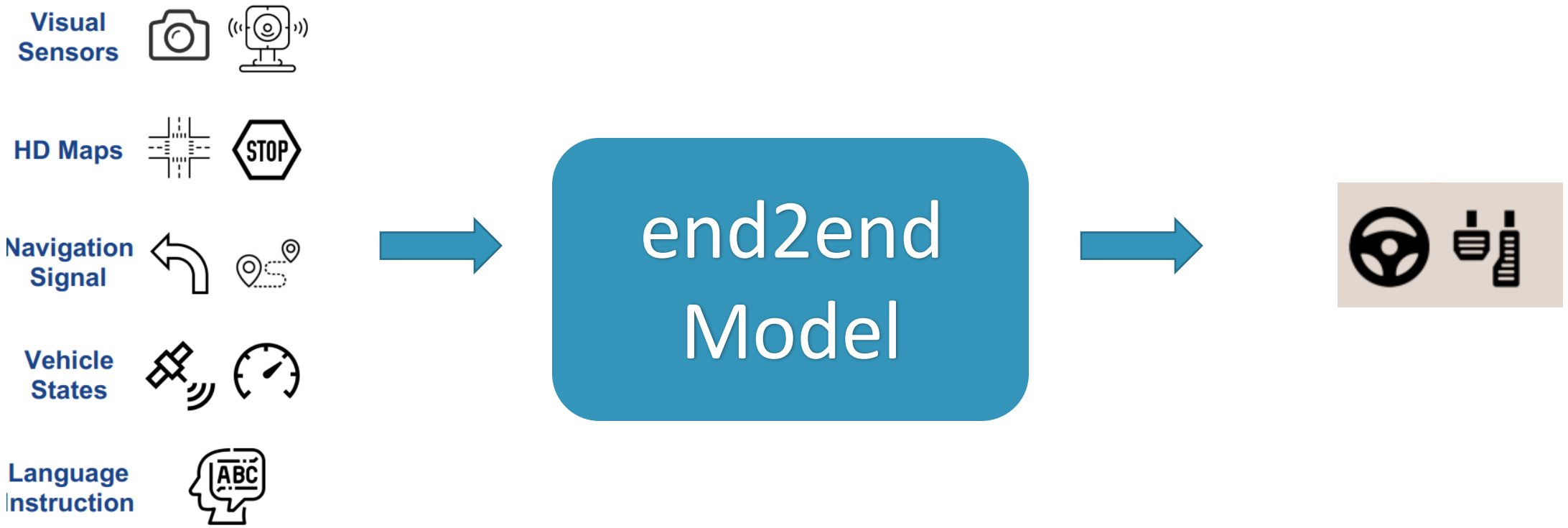
The model generates low-level control actions specific to the platform:

For Self-Driving Cars: Continuous control signals, including steering, throttle, and braking.

For Mobile Robots: Kinematic commands, typically expressed as target velocities and angular velocities.



# End-to-End Navigation




| Output Type | Examples |
|---|---|
| Low-level control | steering, throttle |
| Waypoints | x,y future points |
| Trajectory | discrete future states |
| Affordances | stop/go, lane offset |

# Key Challenges

➢ Massive Data Dependency

Requires immense volumes of diverse and expensive labeled training data.

➢ The "Black Box" Problem

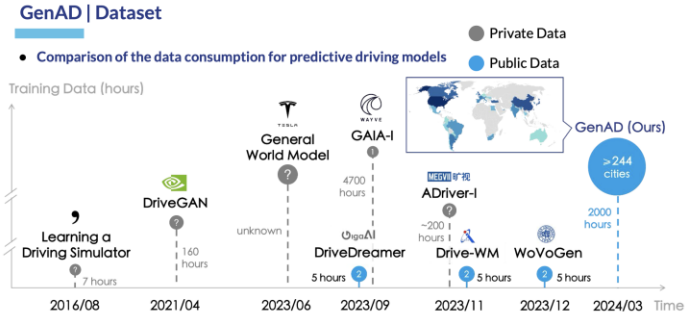
Lack of interpretability and reasoning behind model decisions.

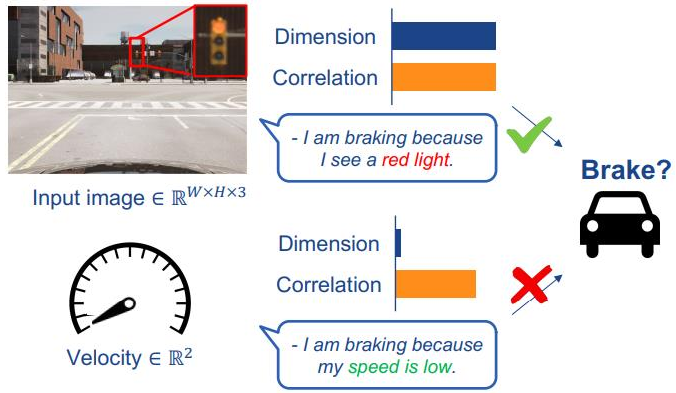
➢ Safety and Verification Hurdles

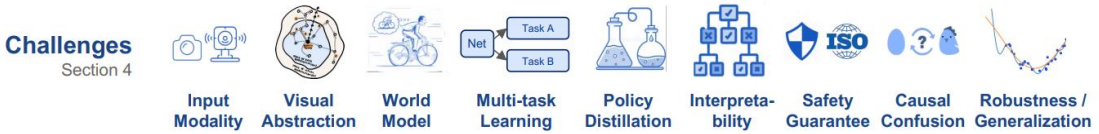
Difficulty in certifying and ensuring reliability in all edge cases.

➢ Generalization Limits

Performance degradation in unseen or highly dynamic environments.



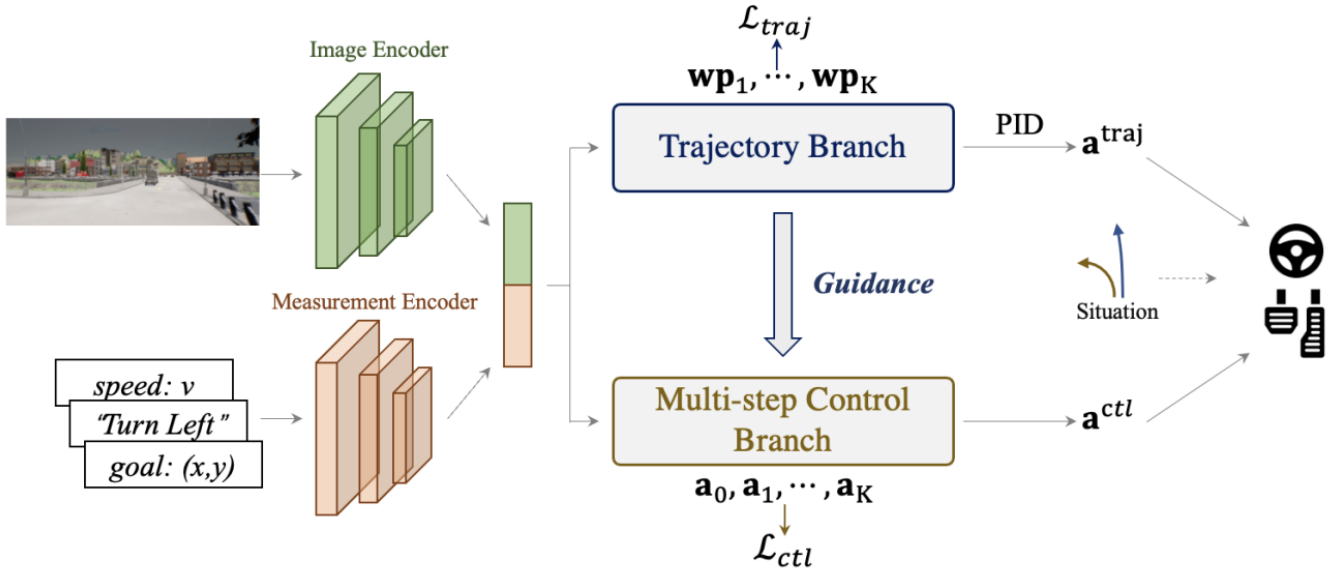



# Three Typical End-to-End Modes

## 1) Direct Perception-to-Control

A neural policy consumes raw sensory observations (e.g., RGB images, lidar, proprioception) and **directly predicts low-level control actions**:

No intermediate maps, no planning module.




**Pipeline**

Camera → CNN/Transformer → actions

$$a_t = f(o_t)$$

**Training**

- Imitation learning (behavior cloning)
- Reinforcement Learning
- Self-supervision (rare)

Source: P. Wu, X. Jia, L. Chen, J. Yan, H. Li, and Y. Qiao, "Trajectory-guided control prediction for end-to-end autonomous driving: A simple yet strong baseline," in NeurIPS, 2022.

# Three Typical End-to-End Modes

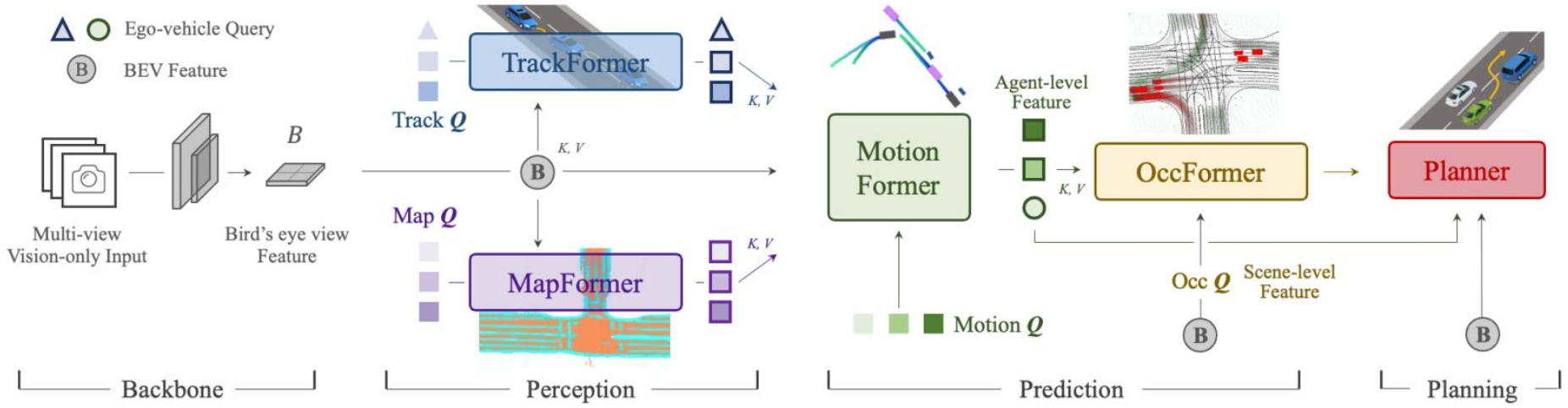
## 2) Perception-to-Intermediate Representation-to-Control

Perception network extracts **spatial + temporal latent representation** (learned or interpretable), then a policy predicts a **trajectory** or **waypoints**, then a controller converts these to commands.

$$o_t \rightarrow z_t \backslash / (BEV / occupancy / waypoints) \rightarrow a_t$$

**Training**

- Imitation learning + self-supervised depth/ego-motion
- Auxiliary losses: segmentation, box prediction, map prediction




Source: Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, "Planning-oriented autonomous driving," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023.

# Three Typical End-to-End Modes

## 2) Perception-to-Intermediate Representation-to-Control

### Types of Intermediate Representations

| Category | Representation |
|---|---|
| Spatial | BEV map, occupancy grid, lane map |
| Geometric | Waypoints, 3D ego-trajectory |
| Agent Modeling | Driver intents, object motion |
| Hybrid | Affordances (e.g., lane position, TTC) |

**Strengths**

1. Much more stable & interpretable than direct E2E
2. Compatible with rule-based safety layers
3. Better generalization across environments
4. Temporal reasoning possible

**Weaknesses**

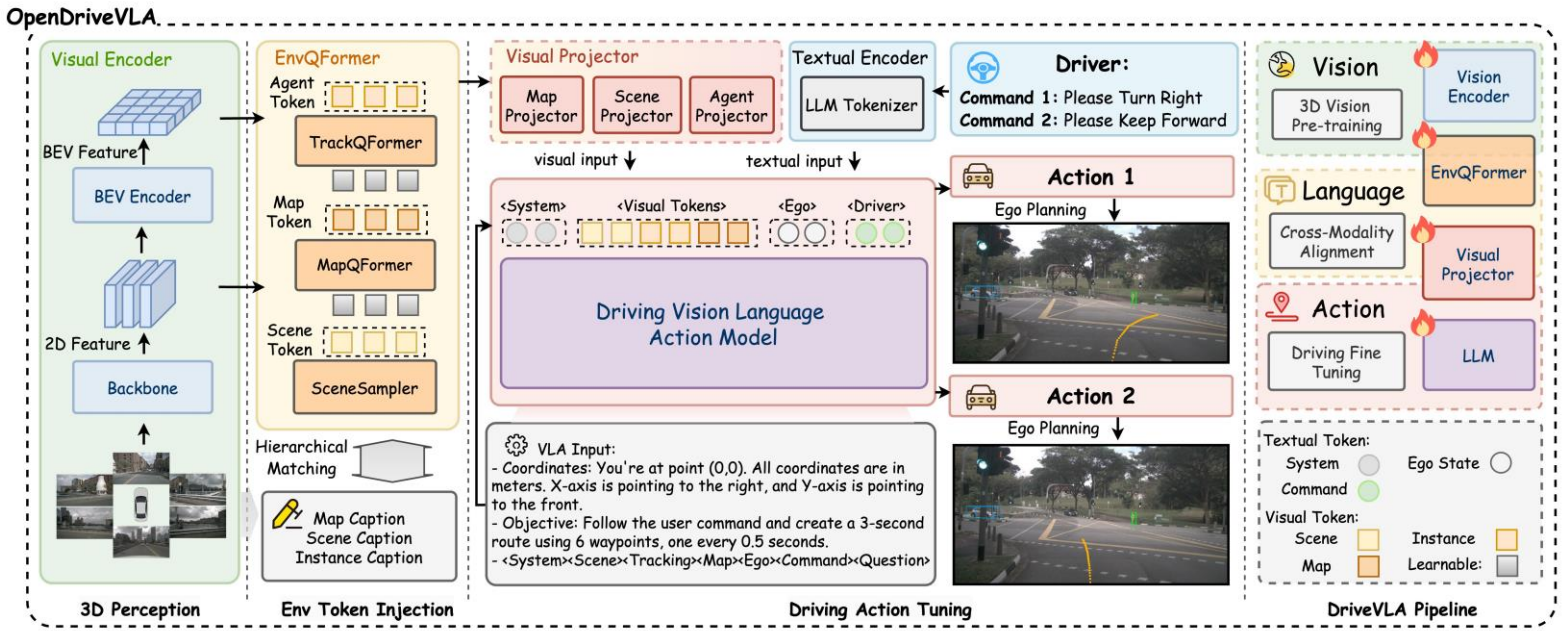
1. Training complexity
2. Requires representation design
3. Can still fail without explicit planning safety

# Three Typical End-to-End Modes

## 3) Perception-to-High-Level Semantics-to-Decision

System extracts **explicit scene semantics** (objects, traffic signals, road topology, intents), then uses rule-based or symbolic reasoning for planning.

$$o_t \rightarrow semantic\_scene \rightarrow high\_level\_policy \rightarrow controller$$




**Semantic Outputs**

- Traffic light state
- Lane & road graph
- Pedestrian/cyclist detection
- Vehicle type & behavior
- Stop signs, speed limits
- Goals, sub-goals, navigation graph

**Decision Layer**

- Planning graph
- Behavior prediction
- Scenario recognition (merging, overtaking)
- Logic + RL hybrid

Source: X. Zhou, X. Han, F. Yang, Y. Ma, and A. C. Knoll, "Opendrivevla: Towards end-to-end autonomous driving with large vision language action model," 2025. [Online]. Available: https://arxiv.org/abs/2503.23463

# Three Typical End-to-End Modes

## 3) Perception-to-High-Level Semantics-to-Decision

**Strengths**

1. Most interpretable
2. Best for regulatory / safety-critical deployment
3. Human-understandable decisions
4. Enables fail-safe logic (HD maps, prior knowledge)

**Weaknesses**

1. Highest engineering effort
2. Semantic errors → cascading failures
3. Requires large labeled datasets

**Usage Examples**

- **Waymo / Cruise / Mobileye** industrial stacks
- **OpenPilot + Nissan ProPILOT Assist**
- Research: semantic route planning, VLN + symbolic planners

**When to Use**

- Safety-critical robotics (AVs, delivery robots)
- Systems requiring explainability & compliance
- Navigation with language commands

# Representations for Navigation

**Three design choices：**

**1.Raw egocentric images**

**2.Intermediate features**

**3.Top-down Bird's-Eye-View (BEV)**

**Core idea**
Navigation : **spatial reasoning** — BEV is spatially aligned.

# Bird's-Eye-View (BEV) Representation

**Definition**

**Top-down feature representation of environment**

**Captures**

- Lane lines
- Drivable area
- Obstacles
- Traffic agents
- Free space & occupancy

Multi-view Input at Time $t$

# Bird's-Eye-View (BEV) Representation

### 1) Unified Spatial Representation

Transforms multi-view camera feeds into a single, coherent top-down map of the environment.

### 2) Core Geometric Foundation

Provides critical 3D spatial understanding, enabling precise distance measurement and object localization.

**Advantages**

### 1) Inherent Geometric Consistency

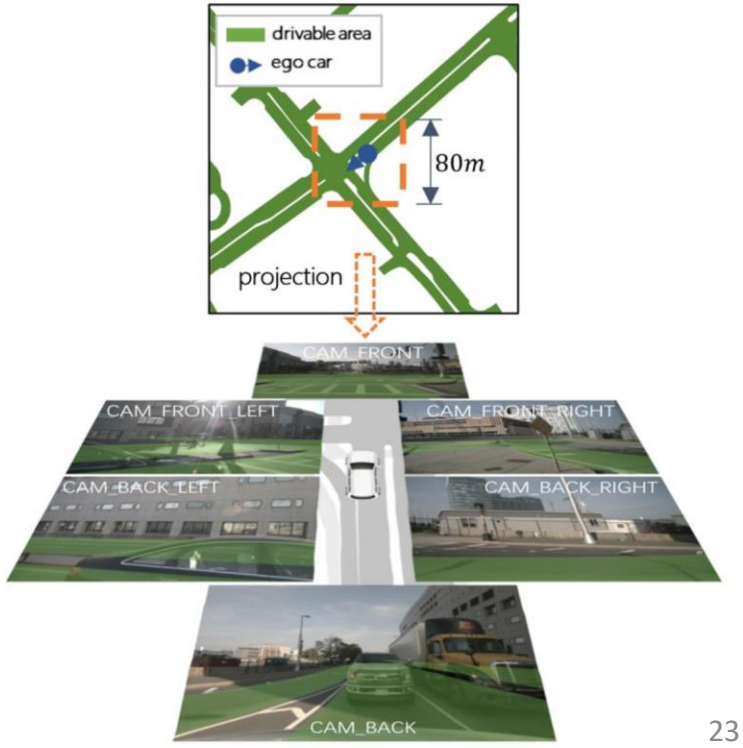
Maintains accurate spatial relationships across all perception inputs.

### 2) Native Multi-Sensor Fusion

Provides a unified canvas for camera, LiDAR, and radar data.

### 3) Intuitive Downstream Planning

Delivers an interpretable scene representation for robust decision-making.



# BEV Generation Methods

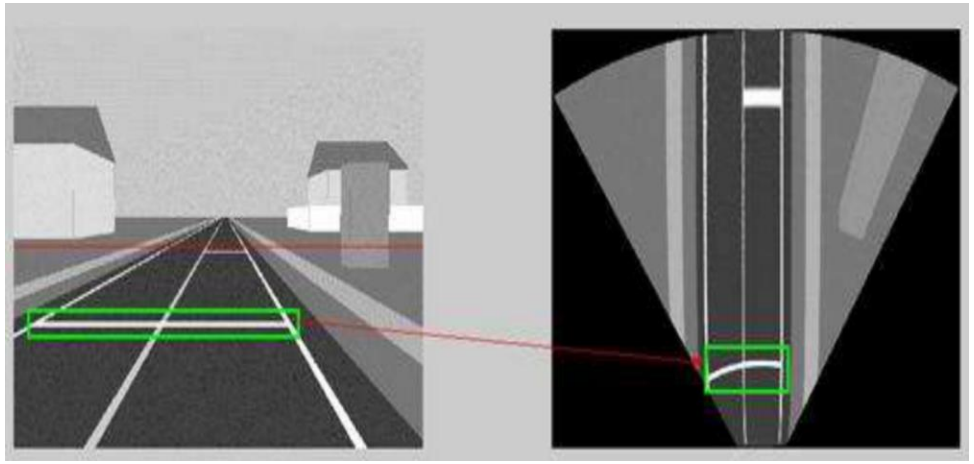
**From sensors to BEV:**

- **LiDAR projection**
- **Depth estimation → point cloud → grid**
- **Inverse perspective mapping (IPM)** from camera

## 1) Geometric Projection

### Inverse Perspective Mapping (IPM):

Transforms image pixels to the BEV space using known camera geometry.



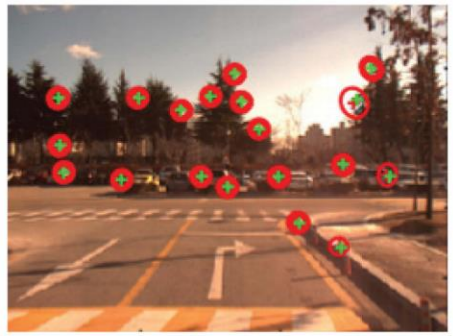
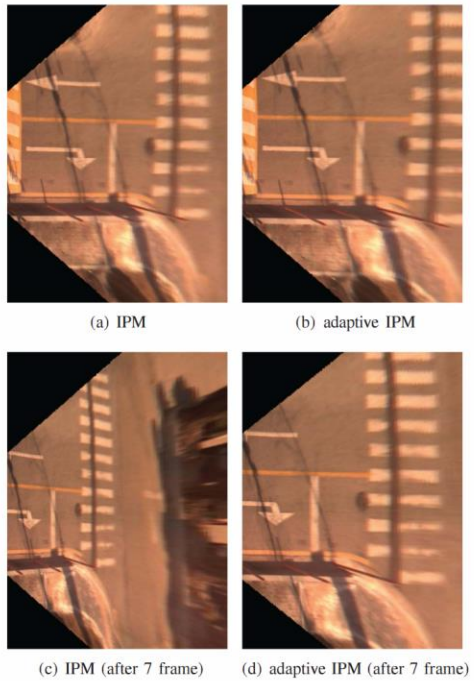


Fig. 5. Feature points derived from visual odometry. The motions of a camera (x, y, z, roll, pitch, yaw) can be calculated using this algorithm.



(a) IPM (b) adaptive IPM

(c) IPM (after 7 frame) (d) adaptive IPM (after 7 frame)

Fig. 6. Experimental results of the data set 1 having a speed bump. There is about 4.1 degrees of difference in the pitch angle between image (**a**) and (**c**). (**c**) and (**d**) are 7 frames after (**a**) and (**b**).

Source: XJ. Jeong and A. Kim, "Adaptive inverse perspective mapping for lane map generation with slam," in 2016 13th International Conference on Ubiquitous Robots and Ambient Intelligence (URAI), 2016, pp. 38–41.

# BEV Generation Methods

**Weaknesses of Geometric Projection:**

- Needs accurate depth or calibration
- Hard with monocular vision

## 2) Deep Learning-Based BEV Generation

Lift-Splat-Shoot (LSS):

To explicitly model 3D space by estimating a depth distribution for every pixel in every camera image before projecting them into a BEV grid.

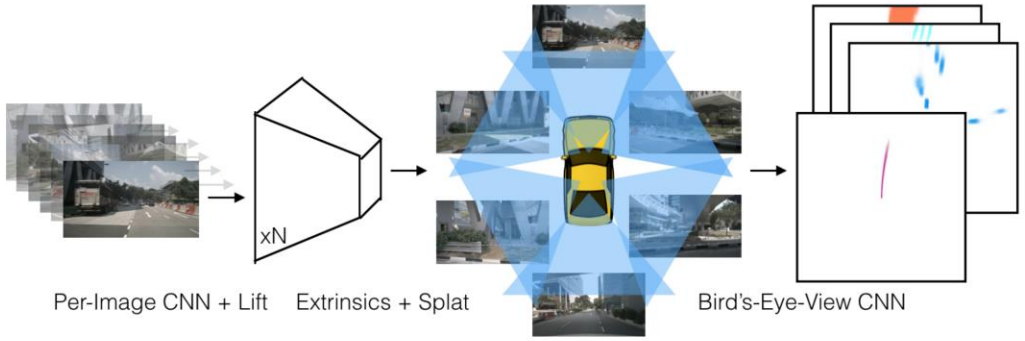



Fig. 4: **Lift-Splat-Shoot Outline** Our model takes as input $n$ images (left) and their

J. Philion and S. Fidler, "Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d," in Proceedings of the European Conference on Computer Vision, 2020.

# BEV Generation Methods

Process of **Lift-Splat-Shoot (LSS)**:

- **Lift (3D Feature Cloud Creation):** For each pixel in each camera image, the network doesn‘t choose a single depth but predicts a depth probability distribution (e.g., over discrete depth bins). This creates a "cloud" of 3D points for each image, where each point has a feature vector and a probability of existing at a certain location.

- **Splat (Voxel Pooling):** This 3D point cloud is then projected into a predefined 3D voxel grid (a 3D grid over the car‘s surroundings). Features from all points that fall into the same voxel are pooled (summed/averaged) together. This step effectively "splats" the points into the grid.

- **Shoot (BEV Map Creation & Downstream Tasks):** The 3D voxel grid is then collapsed along the height dimension (e.g., by taking the maximum value in each column) to create a 2D BEV feature map. This rich BEV map can then be used to "shoot" (perform) downstream tasks like perception, prediction, and planning.

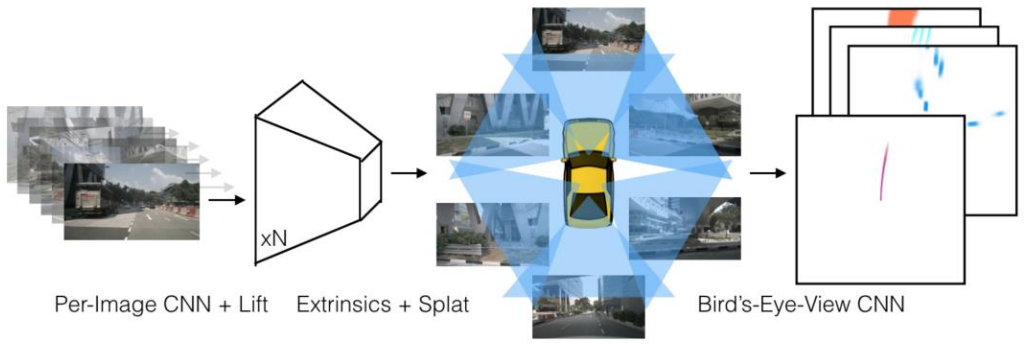



Fig. 4: **Lift-Splat-Shoot Outline** Our model takes as input $n$ images (left) and their

**Lift pixels → 3D → BEV projection**

# BEV Generation Methods

## 2) Deep Learning-Based BEV Generation

### BEVFormer:

To bypass explicit 3D reconstruction and use a transformer architecture to let the model learn how to generate a BEV representation directly from multi-camera inputs, leveraging spatial and temporal information.

- Transformer queries BEV grid
- Temporal fusion (multi-frames)

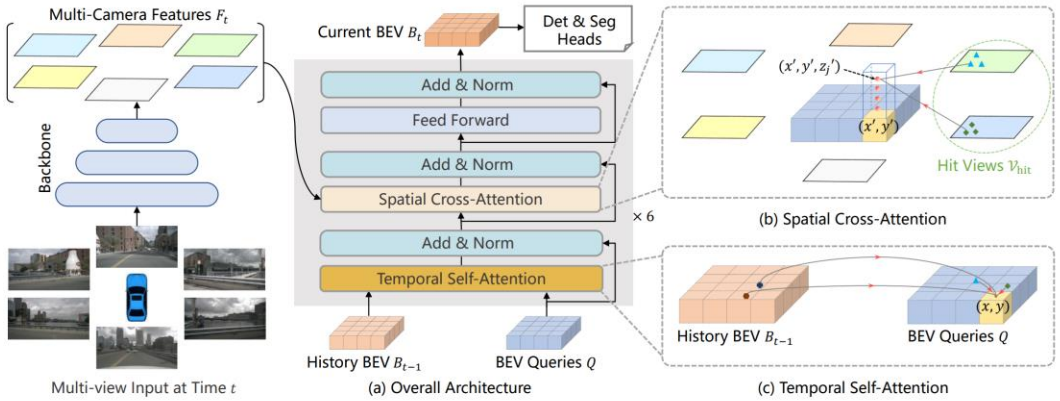



Figure 2: **Overall architecture of BEVFormer.** (a) The encoder layer of BEVFormer contains

Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Y. Qiao, and J. Dai, "Bevformer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers."

# BEV Generation Methods

**Process of BEVFormer**:

- **Define a BEV Query Grid**: The model initializes a grid of learned BEV queries. Each query is a feature vector responsible for a specific physical region in the BEV space (e.g., a 0.5m x 0.5m area).
- **Spatial Cross-Attention**: Each BEV query "looks" back into the multi-camera images to find relevant features. It projects itself into the views of all cameras and uses attention mechanisms to gather and fuse features from the image regions it needs. This is a learned projection, not a fixed geometric one.
- **Temporal Self-Attention**: This is a major advantage over LSS. Each BEV query can also interact with the BEV features from previous timesteps (e.g., the last few seconds). This allows the model to reason about motion, velocity, and occlusion by seeing how the scene has changed over time, dramatically improving perception.
- **Output**: The output of the transformer is a powerful BEV feature map that inherently contains rich spatial and temporal information, ready for task-specific heads (detection, segmentation, etc.).

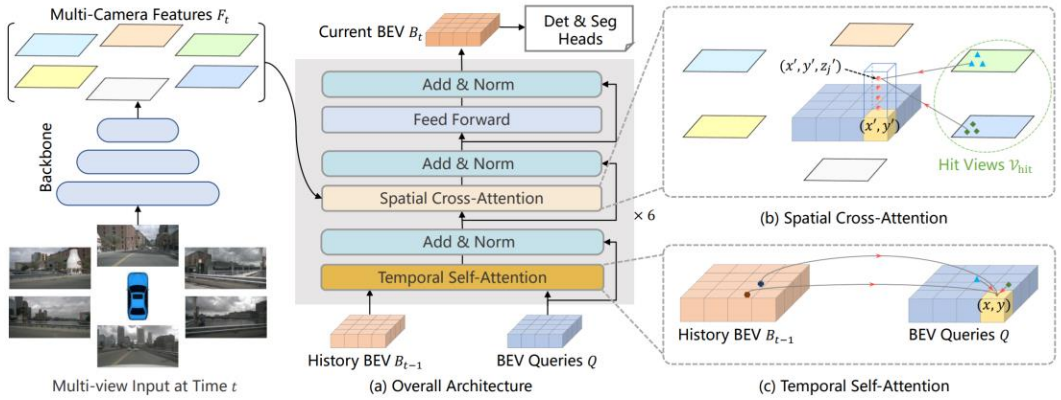
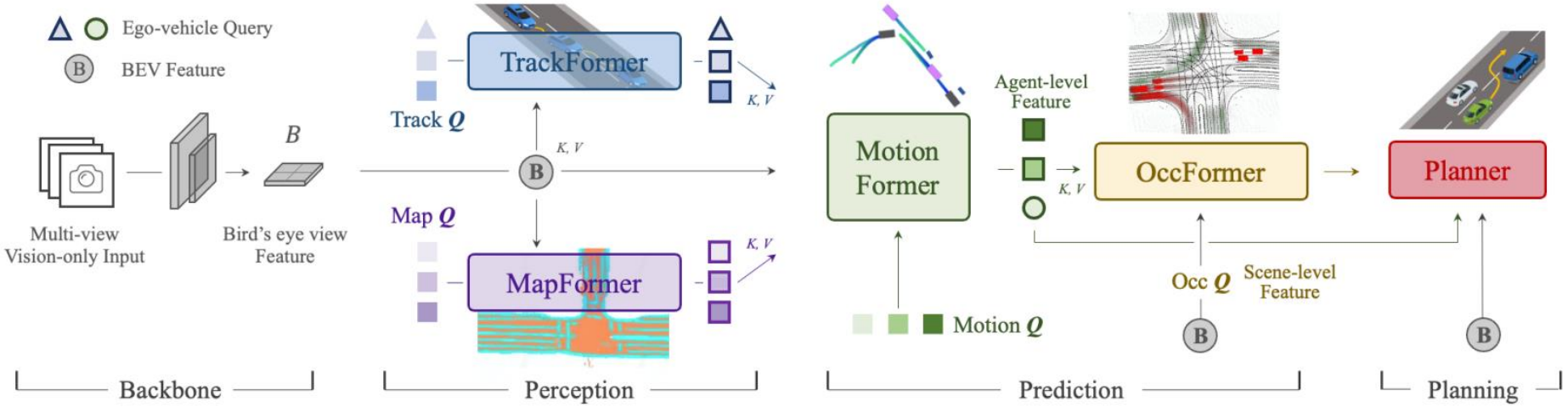



Figure 2: **Overall architecture of BEVFormer.** (a) The encoder layer of BEVFormer contains

# BEV-Inspired End-to-End Driving

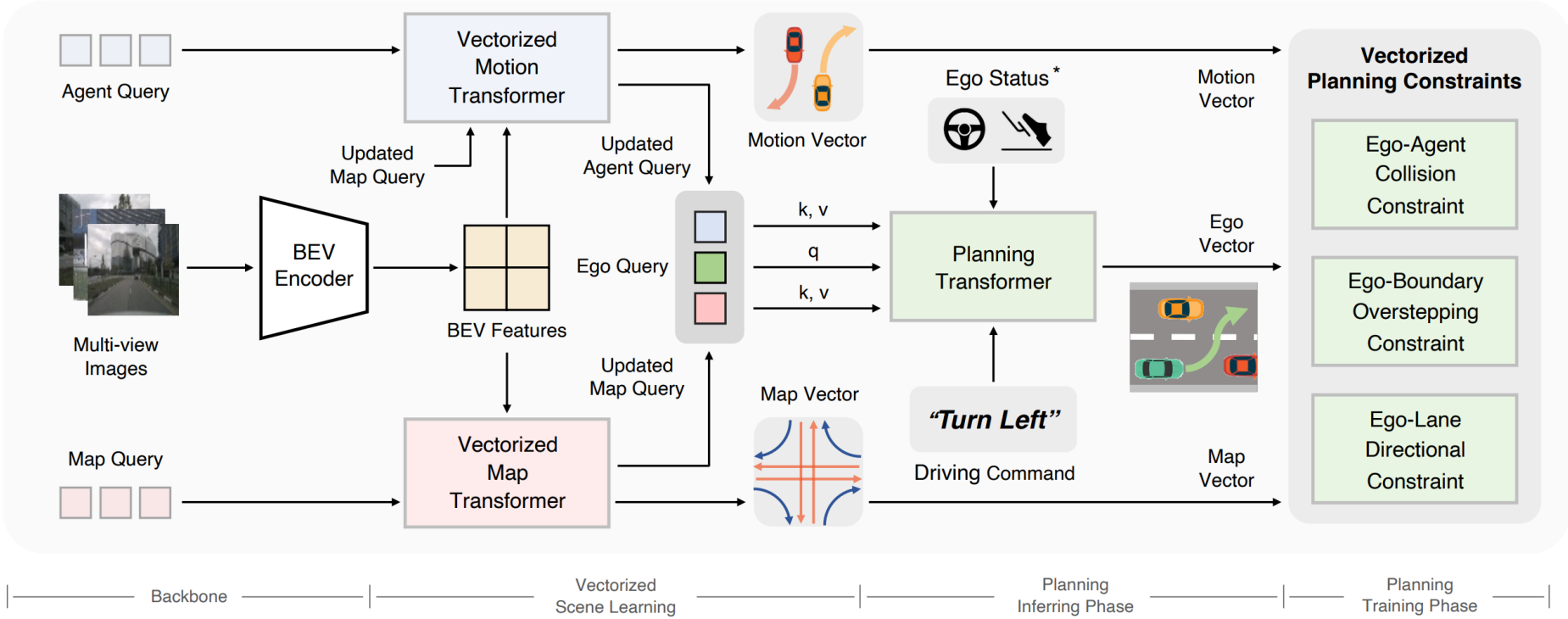
## Case 1: UniAD




Source: Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, "Planning-oriented autonomous driving," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023.

# BEV-Inspired End-to-End Driving

## Case 2: VAD




B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, "Vad: Vectorized scene representation for efficient autonomous driving," ICCV, 2023.

# Benchmark and Evaluation Metrics

**Benchmark Datasets**

**Self-Driving**

- Waymo Open Dataset
- nuScenes
- Argoverse 2
- KITTI BEV tasks

**Indoor Navigation**

- Habitat
- Matterport3D
- Gibson

**Evaluation Metrics**

| Task | Metrics |
|---|---|
| Driving | collision rate, lane accuracy |
| BEV | mIoU, minFDE, ADE |
| Navigation | SPL, success rate |

# Discussion

Open Questions:

1) Should we trust fully end-to-end driving?

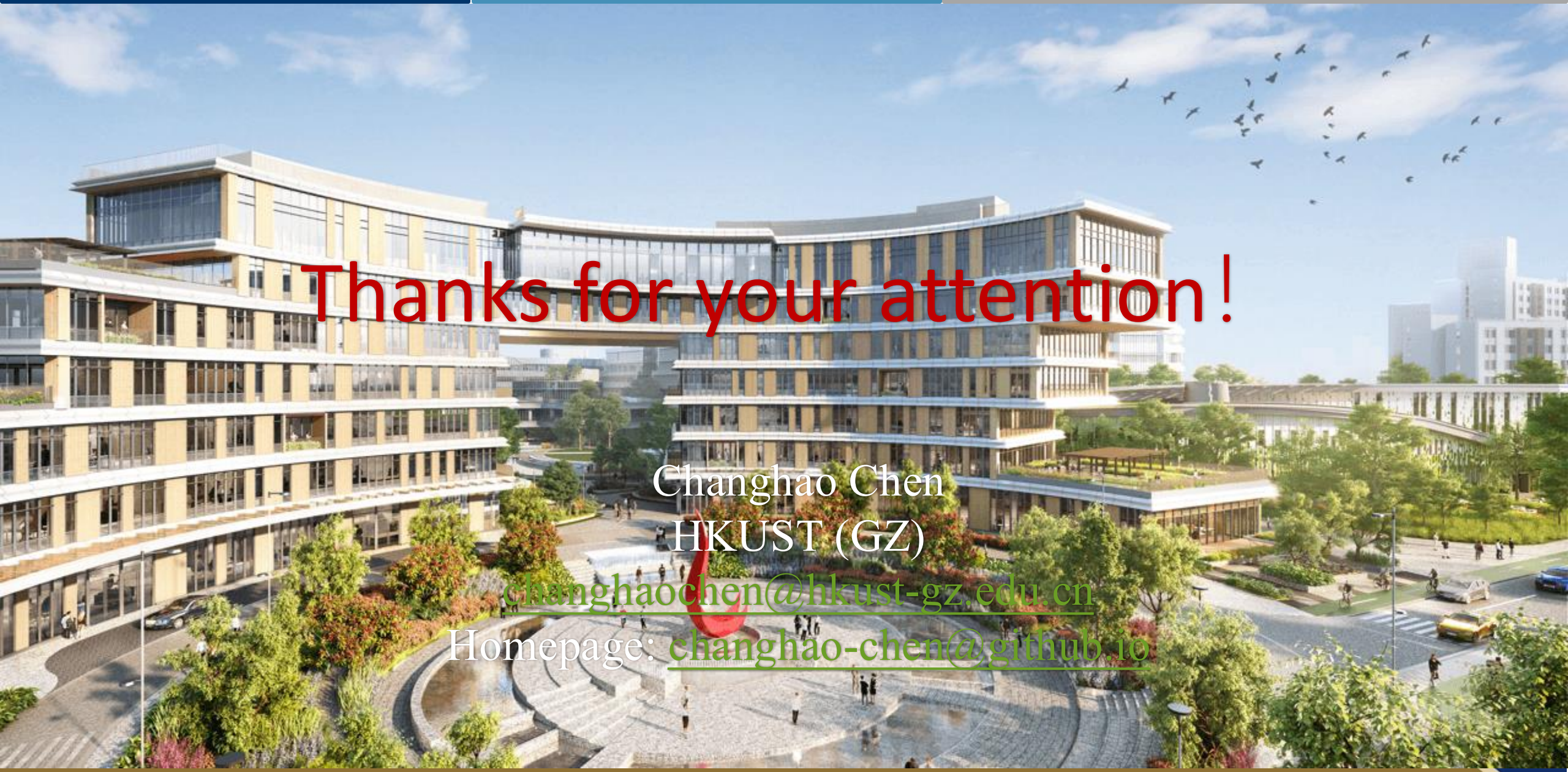
2) Does BEV replace maps?

3) Can robots learn spatial priors without geometry?

# Summary

- End-to-End Navigation
  - Direct sensor input → model → control
  - Advantages: Simplified pipeline, data-driven adaptability, reduced manual work.

- BEV Representation
  - Top-down view from multi-sensor data (Camera, LiDAR, Radar).
  - Methods: Geometric projection, deep learning approaches (Lift-Splat-Shoot, BEVFormer).
  - Benefits:
    - Consistent spatial representation,
    - multi-sensor fusion,
    - improved planning interpretability.



# Thanks for your attention!

Changhao Chen
HKUST (GZ)
changhaochen@hkust-gz.edu.cn
Homepage: changhao-chen@github.io